

本节内容

浮点数

加减运算

本节总览

浮点数的加减运算

类比：十进制科学计数法的加减运算

浮点数加减步骤

①对阶

②尾数加减

③尾数规格化

④舍入

⑤溢出判断

类比：十进制科学计数法的加减运算

浮点数加减运算步骤：

① 对阶

$$9.85211 \times 10^{12} + 9.96007 \times 10^{10}$$

小阶向大阶靠齐
(更有可能得到规格化的尾数)

$$\textcircled{1} 9.85211 \times 10^{12} + 0.0996007 \times 10^{12}$$

② 尾数加减

$$\textcircled{2} 9.9517107 \times 10^{12}$$

尾数天然地已经“规格化”

尾数每左移一位，阶码减1

③ 尾数规格化

③ 如果尾数加减后出现类似 0.0099517×10^{12} 形态，需要“左规”；如果尾数加减后出现类似 99.517107×10^{12} 形态，需要“右规”

尾数每右移一位，阶码加1

④ 尾数的舍入处理

④ 若规定只能保留6位有效尾数，则

- $9.9517107 \times 10^{12} \rightarrow 9.95171 \times 10^{12}$ (多余的尾数直接舍弃)
- 或者， $9.9517107 \times 10^{12} \rightarrow 9.95172 \times 10^{12}$ (若舍弃部分非0，则入1)
- 或者，也可以采用“四舍五入”原则

可以有不同
“舍入”策略

⑤ 溢出判断

⑤ 若规定阶码不能超过99，且运算后阶码超出范围，则发生“溢出”

$$\text{如：} 9.85211 \times 10^{99} + 9.96007 \times 10^{99} = 19.81218 \times 10^{99}$$

尾数规格化并采用四舍五入的原则保留6位尾数，得 1.98122×10^{100}

阶码值超过99，发生溢出

例1：浮点数加法

已知 float X = 1.5, Y = 0.5 符合 IEEE 754 标准。分析计算机实现 float Z = X + Y 的过程。

X → + 1.1 × 2⁰ 0 01111111 100000000000000000000000
Y → + 1.0 × 2⁻¹ 0 01111110 000000000000000000000000

尾数高位隐含1

+1.100000000000000000000000

← X 尾数原码

+1.000000000000000000000000

← Y 尾数原码

① 对阶 (原则：小阶向大阶靠齐) X_阶 - Y_阶 = 1, 说明 Y 的阶码更小, 应将 Y 的尾数右移 1 位, 阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X 尾数不变 →

+1.100000000000000000000000

Y 尾数右移 1 位 →

+0.100000000000000000000000

IEEE 规定右移移出的 bit 至少额外保留 3 位

② 尾数加减 (原码的加减运算)

原码加法, 得 Z 尾数原码 →

+10.0000000000000000000000

③ 尾数规格化

Z_阶 = 大阶 + 1
= 01111111 + 1
= 10000000

尾数右规 (右移 1 位)

Z 尾数原码 →

+1.0000000000000000000000

④ 尾数的舍入处理

舍弃多余尾数 (0 舍 1 入)

⑤ 溢出判断 → 阶码不全为 1、也不全为 0, 说明未发生上溢、下溢

【符, 阶, 尾】 = 【0 10000000 000000000000000000000000】

Z = + 1.0 × 2¹ = 2

例2：浮点数减法

已知 float $X = 0.125$, $Y = 0.5$ 符合 IEEE 754 标准。分析计算机实现 float $Z = X - Y$ 的过程。

$X \rightarrow +1.0 \times 2^{-3}$ 0 01111100 000000000000000000000000

$Y \rightarrow +1.0 \times 2^{-1}$ 0 01111110 000000000000000000000000

尾数高位隐含1

+1.000000000000000000000000

← X 尾数原码

+1.000000000000000000000000

← Y 尾数原码

① 对阶 (原则：小阶向大阶靠齐) $X_{\text{阶}} - Y_{\text{阶}} = -2$, 说明 X 的阶码更小, 应将 X 的尾数右移 2 位, 阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X 尾数右移 2 位 →

+0.010000000000000000000000

Y 尾数不变 →

+1.000000000000000000000000

IEEE 规定右移移出的 bit 至少额外保留 3 位

② 尾数加减 (原码的加减运算)

原码减法, 得 Z 尾数原码 →

-0.110000000000000000000000

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} - 1$
 $= 01111110 - 1$
 $= 01111101$

Z 尾数原码 →

-1.100000000000000000000000

尾数左规 (左移 1 位)

④ 尾数的舍入处理

舍弃多余尾数 (0 舍 1 入)

-1.100000000000000000000000

⑤ 溢出判断 → 阶码不全为 1、也不全为 0, 说明未发生上溢、下溢

【符, 阶, 尾】 = 【1 01111101 100000000000000000000000】

$Z = -0.375$

例3：浮点数减法

已知 float $X = 10.5$, $Y = 120.625$, 符合 IEEE 754 标准。分析计算机实现 float $Z = X - Y$ 的过程。

$X \rightarrow +1.0101 \times 2^3$ 0 10000010 010100000000000000000000
 $Y \rightarrow -1.111000101 \times 2^6$ 0 10000101 111000101000000000000000

尾数高位隐含1

+1.010100000000000000000000	← X _尾
+1.111000101000000000000000	← Y _尾

① 对阶 (原则：小阶向大阶靠齐) $X_{\text{阶}} - Y_{\text{阶}} = -3$, 说明 X 的阶码更小, 应将 X 的尾数右移3位, 阶码对齐

硬件实现原理：原码转补码，加/减后再转回原码

X尾数右移3位 →

+0.001010100000000000000000

Y尾数不变 →

+1.111000101000000000000000

IEEE规定右移移出的bit至少额外保留3位

② 尾数加减 (原码的加减运算)

原码减法, 得 $Z_{\text{尾}} \rightarrow$

-1.101110001000000000000000

尾数天然地已经规格化, 无需移位

③ 尾数规格化

$Z_{\text{阶}} = \text{大阶} + 0$
 $= 10000101 + 0$
 $= 10000101$

$Z_{\text{尾}} \rightarrow$

-1.101110001000000000000000

④ 尾数的舍入处理

舍弃多余尾数 (0舍1入)

-1.101110001000000000000000

⑤ 溢出判断 → 阶码不全为1、也不全为0, 说明未发生上溢、下溢

【符, 阶, 尾】 = 【1 10000101 101110001000000000000000】

$Z = -1.101110001 \times 2^6 = -1101110.001\text{B} = -110.125$